Guía de Actividades Práctico-Experimentales Nro. 001

1. Datos Generales

Asignatura	Estructura de Datos
Integrantes	- Freddy Ordoñez - Yimmy Angulo - Richard Guaman - Erick Jahirs - Joaquin Moscol
Ciclo	3ro A
Unidad	Estructuras de datos estáticas (arreglos, conjuntos, registros, archivos) y estructuras de datos dinámicas (listas, pilas y colas)
Resultado de aprendizaje de la unidad	R1. Implementa algoritmos simples de búsqueda y explica las diferencias en el orden de complejidad computacional, bajo los principios de solidaridad.
Práctica Nro.	01
Título de la Práctica	Laboratorio inicial en IDE para medir bytes en tipos primitivos vs objetos.
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 01 y viernes 02 de octubre 2026
Horario	Jueves 01 de octubre 10:30 – 11:30 Viernes 02 de octubre 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

- **Modelar** Tipos de Datos Abstractos (TDA) simples utilizando clases y constructores en JavaScript moderno.
- **Medir** empíricamente el impacto en la memoria Heap (complejidad espacial S(n) al procesar estructuras de datos basadas en tipos primitivos frente a arreglos de objetos masivos.

- **Evaluar** mediante el análisis del recolector de basura (Garbage Collector) las decisiones de diseño físico y su impacto en la sostenibilidad computacional (Green Computing).

3. Materiales y Reactivos

- Software Node.js (versión LTS 20.x o superior) instalado en el sistema.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación técnica oficial (ABI).
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Procedimiento / Metodología

Enfoque PID-2025:

- **ABP (Aprendizaje Basado en Problemas):** El equipo es contratado por un proyecto de geolocalización satelital sostenible. Deben cargar $N = 10^6$ coordenadas en memoria. El problema radica en que instanciar 10^6 objetos colapsa los límites del entorno. Se debe diseñar y medir la alternativa más eficiente.
- **ABI (Aprendizaje Basado en Investigación):** Los estudiantes deberán investigar cómo el motor de JavaScript (V8) reserva memoria contigua vs. memoria dinámica.
- **Modalidad: Individual y discusión grupal posterior.**

Tarea 1: Preparación del Entorno de Medición (Profiler)

Para medir la huella de memoria en JavaScript, utilizaremos el módulo integrado process.

1. **Cree un nuevo directorio para el proyecto y abra su editor de texto.**
2. **Cree un archivo denominado profiler.js.**
3. **Implemente la función de medición de memoria RAM (Heap):**

```
/**
 * Tarea 1: Utilidad para medir la memoria Heap en Megabytes (MB).
 * El Heap es el espacio de memoria dinámico donde V8 almacena objetos y variables.
 */
function getMemoryUsage() {
    const memoryData = process.memoryUsage();
    // Convertimos de bytes a Megabytes (MB) usando notación matemática estándar
    const heapUsedMB = Math.round(memoryData.heapUsed / 1024 / 1024 * 100) / 100;
    return heapUsedMB;
}
```

module.exports = { getMemoryUsage };



Tarea 2: Enfoque 1 - Modelado basado en Arreglo de Objetos (TDA)

En esta tarea, crearemos N objetos. Esta es la representación de más alto nivel de abstracción.

1. **Cree un archivo llamado lab01_objetos.js.**
2. **Incluya el siguiente código, el cual creará $1,000,000$ de instancias de coordenadas.**

```
const { getMemoryUsage } = require('./profiler');

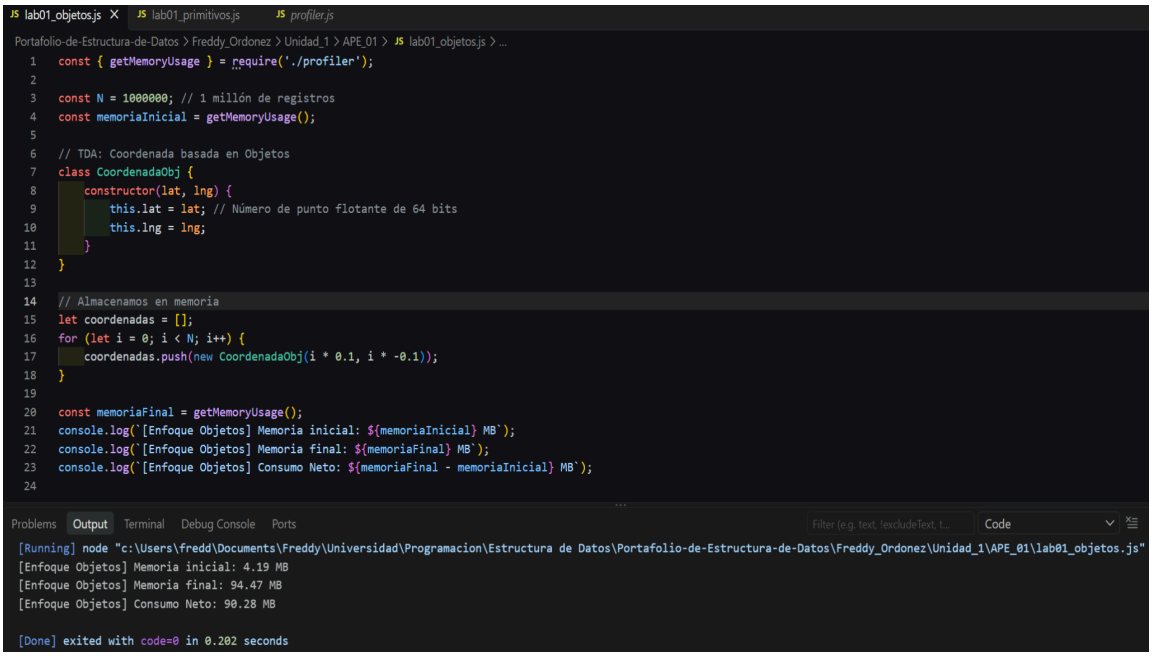
const N = 1000000; // 1 millón de registros
const memoriaInicial = getMemoryUsage();

// TDA: Coordenada basada en Objetos
class CoordenadaObj {
    constructor(lat, lng) {
        this.lat = lat; // Número de punto flotante de 64 bits
        this.lng = lng;
    }
}

// Almacenamos en memoria
let coordenadas = [];
for (let i = 0; i < N; i++) {
    coordenadas.push(new CoordenadaObj(i * 0.1, i * -0.1));
}

const memoriaFinal = getMemoryUsage();
console.log(`[Enfoque Objetos] Memoria inicial: ${memoriaInicial} MB`);
console.log(`[Enfoque Objetos] Memoria final: ${memoriaFinal} MB`);
console.log(`[Enfoque Objetos] Consumo Neto: ${memoriaFinal - memoriaInicial} MB`);
```

3. ***Ejecute en su terminal: node lab01_objetos.js y anote el resultado neto.***



Tarea 3: Enfoque 2 - Modelado basado en Arreglos de Primitivos paralelos (Data-Oriented Design)

1. **Cree un archivo llamado lab01_primitivos.js.**
2. **Implemente la estructura de datos desacoplando la abstracción del objeto a arreglos paralelos unidimensionales fuertemente tipados (Float64Array).**

```
const { getMemoryUsage } = require('./profiler');

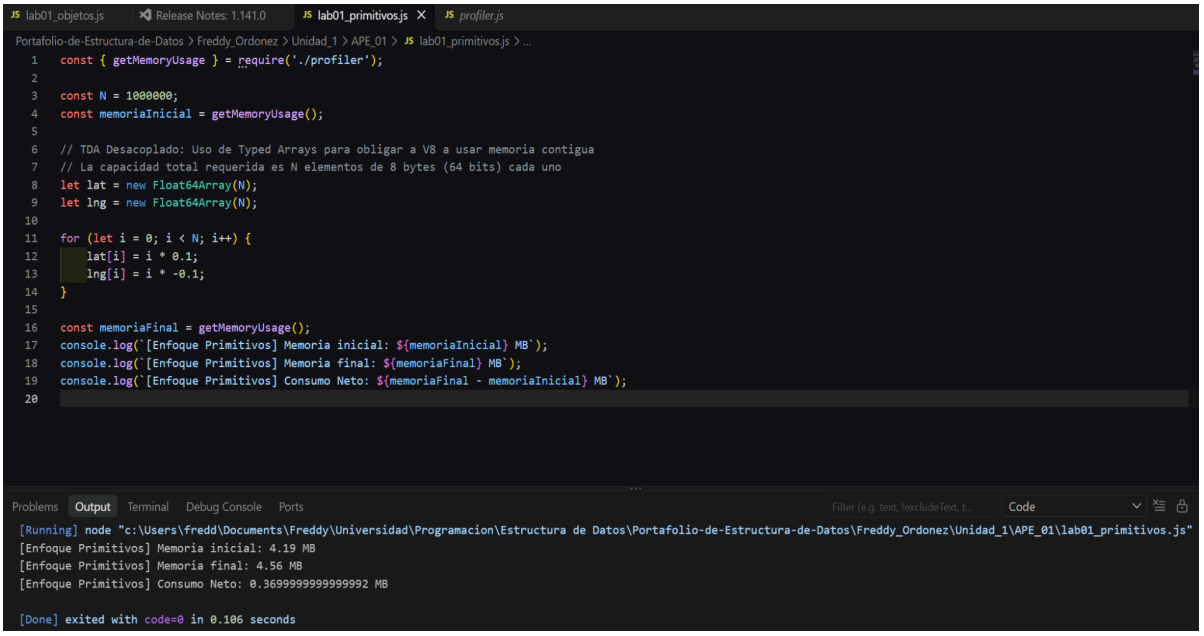
const N = 1000000;
const memoriaInicial = getMemoryUsage();

// TDA Desacoplado: Uso de Typed Arrays para obligar a V8 a usar memoria contigua
// La capacidad total requerida es N elementos de 8 bytes (64 bits) cada uno
let lat = new Float64Array(N);
let lng = new Float64Array(N);

for (let i = 0; i < N; i++) {
    lat[i] = i * 0.1;
    lng[i] = i * -0.1;
}

const memoriaFinal = getMemoryUsage();
console.log(`[Enfoque Primitivos] Memoria inicial: ${memoriaInicial} MB`);
console.log(`[Enfoque Primitivos] Memoria final: ${memoriaFinal} MB`);
console.log(`[Enfoque Primitivos] Consumo Neto: ${memoriaFinal - memoriaInicial} MB`);
```

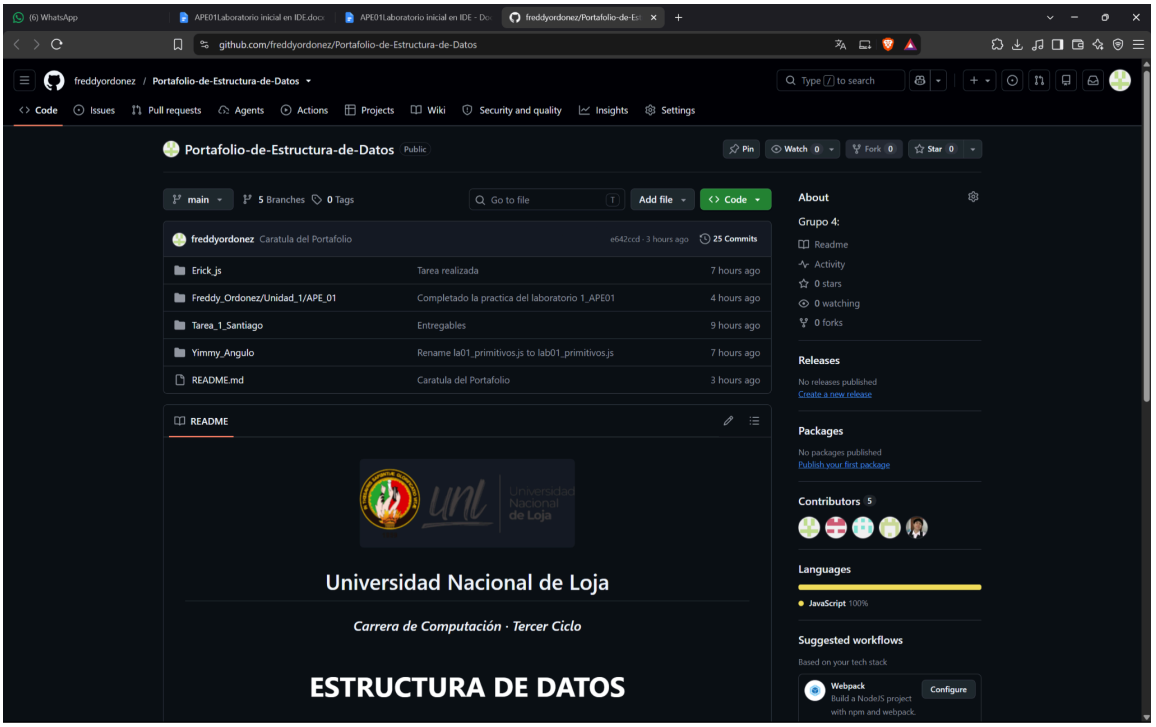
3. *Ejecute en su terminal: node lab01_primitivos.js y anote el resultado.*



Repositorio de Github Grupal:

https://github.com/freddyordonez/Portafolio-de-Estructura-de-Datos.git

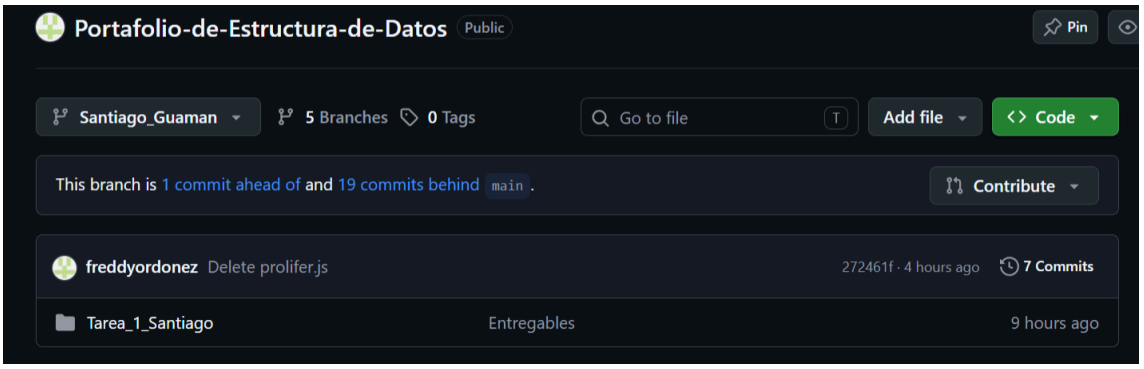
Capturas de la Rama Principal



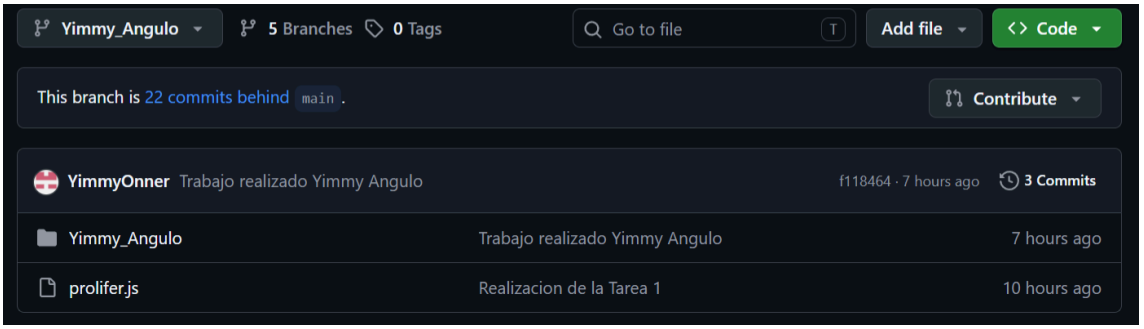
Capturas de la Rama de Freddy Ordoñez



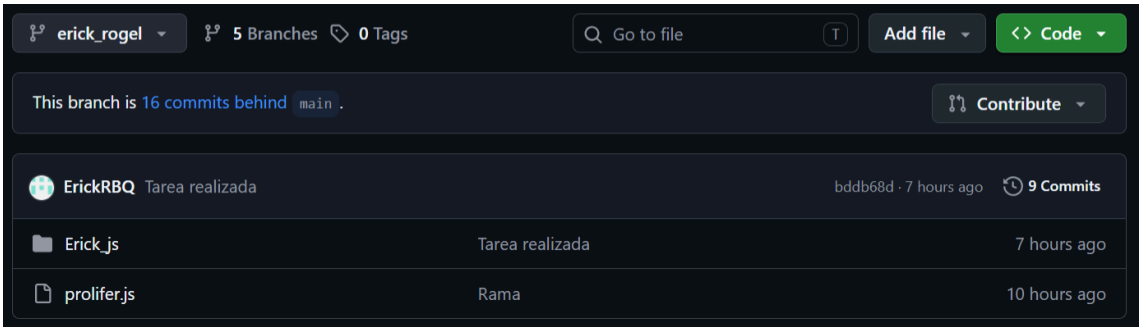
Capturas de la Rama de Richard Guaman



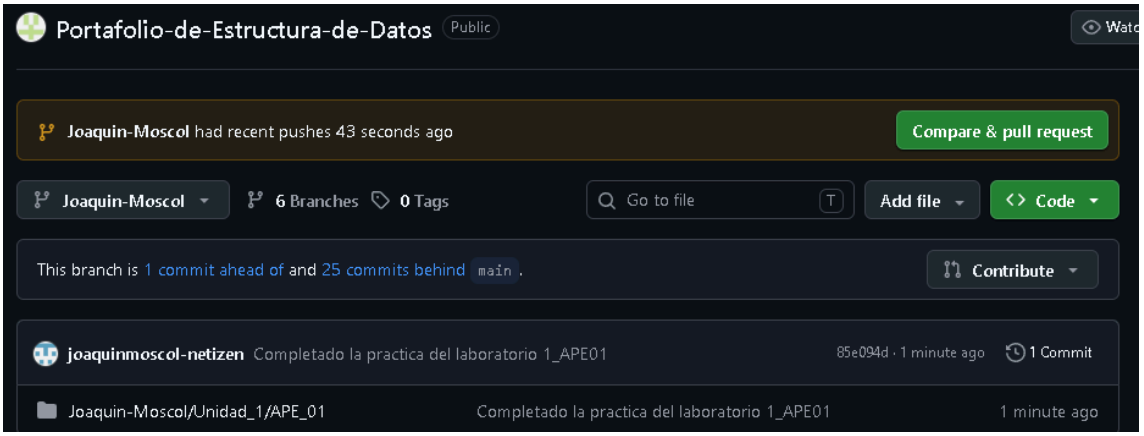
Capturas de la Rama de Yimmy Angulo



Capturas de la Rama de Erick Jahirs



Capturas de la Rama de Joaquin Moscol



Tarea 4: Investigación y Cálculo de Complejidad

Basado en los resultados, el estudiante debe aplicar ABI para investigar el porqué de la diferencia matemática.

1. **Busque en la documentación oficial de V8 cómo se empaquetan las propiedades en los *Hidden Classes* de JavaScript.**

El **motor V8** optimiza el rendimiento separando el diseño de los datos en lugar de buscar propiedades por su nombre en tablas hash lentas, utiliza clases ocultas **(*Hidden Classes*)** para asignarles una posición fija en la memoria, permitiendo un acceso directo a los valores.

Por ejemplo, si defines dos vehículos con las mismas características en el mismo orden:

```
1 const auto1 = { marca: "Toyota", año: 2022 };
2 const auto2 = { marca: "Honda", año: 2024 };
```

Como ambos objetos comparten idéntica estructura y orden (**marca** y luego **año**), el motor V8 emplea un único *Hidden Class* para los dos. Esto significa que V8 sabe exactamente en qué posición exacta de la memoria se encuentra la marca o el año de cualquiera de los autos, guardando los valores de forma directa sin necesidad de buscar nombres de propiedades uno por uno.

2. **Calcule teóricamente en LaTeX el espacio asintótico. Sabiendo que un Float64 pesa 8 bytes, demuestre por qué.**

1 Análisis del Espacio Asintótico

Para entender cómo crece el uso de memoria en las estructuras de datos, se analiza el comportamiento de un arreglo básico. Si definimos como n a la cantidad total de elementos que se guardan en una estructura de tipo `Float64`, podemos calcular cuánta memoria ocupan.

1.1 Demostración del Tamaño en Memoria

Un dato de tipo `Float64` utiliza 64 bits de espacio. Como cada byte equivale a 8 bits, el tamaño real que ocupa cada elemento individual se obtiene dividiendo esos valores:

$$64 \text{ bits} = \frac{64}{8} = 8 \text{ bytes}$$

Por lo tanto, si la estructura almacena n elementos de este tipo, la memoria total necesaria se calcula multiplicando la cantidad de elementos por el espacio de cada uno:

$$M(n) = 8n \text{ bytes}$$

1.2 Tasa de Crecimiento y Comportamiento

Para conocer cómo evoluciona este consumo a medida que la cantidad de datos aumenta de forma masiva, se aplica el análisis asintótico. En este estudio se descarta el número constante 8, ya que lo verdaderamente importante es observar cómo crece la memoria de manera proporcional conforme n se hace más grande.

Si duplicamos la cantidad de elementos, la memoria utilizada también se duplica exactamente, lo que demuestra un avance completamente constante y directo. Aplicando la notación formal de la cota Θ *(Theta)*:

$$M(n) \in \Theta(n)$$

Como conclusión, el espacio necesario para almacenar los datos crece de forma lineal:

$$\boxed{\Theta(n)}$$

2 Análisis Matemático / Teórico: Explicación del *Overhead* con Notación Big-O

Cuando se trabaja con tipos de datos estructurados u objetos en lugar de tipos primitivos puros, el sistema añade metadatos adicionales de control (cabeceras de objetos, apuntadores a la clase o referencias de memoria). Este costo adicional se conoce como *overhead*.

Si el tamaño de un objeto primitivo es C y el *overhead* fijo por cada objeto o la estructura contenedora es K, la función de memoria total se modela como:

$$M_{\text{objeto}}(n) = Cn + K$$

Al aplicar la notación asintótica Big-O, las constantes multiplicativas y los términos independientes (el *overhead* fijo K) se desprecian para evaluar el comportamiento límite:

$$M_{\text{objeto}}(n) \in \mathcal{O}(n)$$

A pesar de que el *overhead* incrementa significativamente la constante de proporcionalidad en la práctica, asintóticamente el crecimiento sigue siendo lineal $\mathcal{O}(n)$.

3 Medición y Tabulación de Datos (Primitivos vs. Objetos)

Los datos experimentales obtenidos de la ejecución de las pruebas en entorno Node.js reflejan el impacto directo del almacenamiento basado en valores primitivos frente al uso de objetos. A continuación se detallan los resultados empíricos registrados:

Enfoque	Mem. Inicial	Mem. Final	Consumo Neto
Primitivos (`primitivos.js`)	4.19 MB	4.62 MB	0.43 MB
Objetos (`objetos.js`)	4.19 MB	94.48 MB	90.29 MB

Como se muestra en la tabla, usar objetos gasta mucha más memoria. Esto pasa porque cada objeto guarda información extra y referencias además del dato en sí, haciendo que el consumo de espacio sea mucho mayor en comparación con los datos primitivos más sencillos.

Link del documento hecho en Latex:

https://www.overleaf.com/project/6ac82dc42ec5141c86fbc992

5. Resultados esperados:

- El estudiante obtendrá evidencia empírica en consola demostrando que el manejo de Tipos de Datos Abstractos complejos mediante instanciación masiva de objetos genera un *overhead* significativo en la memoria (*heap*), incrementando la latencia del recolector de basura. Se espera una diferencia de consumo sustancial a favor de los Typed Arrays (Float64Array).

6. Preguntas de Control:

- Responda las siguientes preguntas reflexionando sobre las causas fundamentales de los fenómenos computacionales observados:

1. **Causa Material:** ¿De qué está constituido fundamentalmente un objeto en la memoria del entorno de ejecución (V8), y en qué difiere la "materia" de un objeto de la de un valor primitivo continuo en un Float64Array?

   En el motor V8, un objeto está constituido fundamentalmente por un mapa oculto que define su estructura y la posición de sus propiedades y por punteros o referencias que apuntan a donde realmente se guardan los valores en la memoria. Al ser estructuras dinámicas que pueden cambiar de forma, sus datos no están ordenados de manera continua, sino dispersos y accesibles mediante estas referencias.

   En contraste, un **Float64Array** almacena los datos como un bloque de memoria plano y continuo, donde los números en bruto se ubican uno detrás de otro de forma estricta. A diferencia del objeto, carece de metadatos o punteros complejos, permitiendo que la computadora acceda y procese la información directamente a nivel de hardware con máxima eficiencia

2. **Causa Eficiente:** Al observar el consumo de memoria, ¿es el intérprete de JavaScript el verdadero artífice (causa eficiente) del desperdicio de recursos, o reside la responsabilidad primaria en la decisión arquitectónica del desarrollador?

   La responsabilidad primaria recae en la decisión arquitectónica del desarrollador, y el intérprete de JavaScript es solo la causa instrumental. V8 es quien físicamente reserva la memoria, pero lo hace obedeciendo el código que recibe. En términos tradicionales, la causa eficiente es el agente que origina el cambio, y aquí ese agente es el programador que decide cómo modelar los datos. V8 ejecuta esa decisión sin elegirla.

   Este trabajo lo demuestra con el mismo problema ($N = 10^6$ coordenadas) y el mismo motor. El arreglo de objetos (CoordenadaObj) consumió 90.28 MB, porque cada instancia carga su cabecera, su Hidden Class y un puntero en el arreglo contenedor. Los Float64Array paralelos requieren teóricamente solo 16n bytes (unos 16 MB en un bloque contiguo). El heapUsed marcó apenas 0.37 MB porque ese almacenamiento vive fuera del heap de V8. Como el motor fue el mismo en ambos casos, la diferencia de unas 5.6 veces solo

puede venir de la estructura de datos elegida, y por eso el responsable del desperdicio es quien la elige, no quien la ejecuta.

3. **Causa Final:** ¿Cuál es el propósito último de abstraer los datos en la programación orientada a objetos si, en su esencia más fundamental, toda optimización crítica requiere retornar a la disposición contigua de bits y bytes en la memoria?

   La causa final de la abstracción en la Programación Orientada en Objetos no es ocultar la memoria para siempre, sino gestionar la complejidad separando el "qué" del "cómo". Aunque a nivel de hardware todo se reduce a bytes contiguos, la abstracción cumple un propósito práctico fundamental al permitirnos trabajar con conceptos del dominio (como Cliente o Membresia) en lugar de lidiar con direcciones de memoria.

   Esta separación se apoya en pilares como la encapsulación (controlar el estado), la mantenibilidad (cambiar la lógica interna sin romper el código externo), la modularidad y la reutilización. Gracias a esto, un ejemplo como cliente.agregarMembresia(membresia) nos permite pensar en el problema lógico mientras la máquina virtual se encarga de las referencias y los bytes por debajo.

   Por tanto, la abstracción no busca reemplazar la realidad física del computador, sino evitar que sea una carga cognitiva permanente. Su causa final es optimizar el pensamiento humano antes que el acceso a memoria, reservando el acercamiento al hardware únicamente para cuando el rendimiento extremo lo exija.

4. **Esencia y Existencia:** Si dos estructuras de datos (como el Enfoque 1 y el Enfoque 2) resuelven exactamente el mismo problema lógico arrojando los mismos resultados, pero existen físicamente ocupando volúmenes de energía y memoria drásticamente distintos, ¿son en esencia la misma solución? Argumente.

   Sí, en esencia son la misma solución, pero no en existencia. La esencia es lo que la solución es: guardar un millón de coordenadas (latitud y longitud) para poder consultarlas. La existencia es cómo esa solución se materializa en la máquina, y ahí son muy distintos: uno ocupa 90.28 MB y el otro, en teoría, unos 16 MB.

   Es como llevar la misma carga en un camión lleno de cajas sueltas o en una sola caja bien empacada. La carga es idéntica, pero el costo del viaje no. Lo mismo pasa en el trabajo: ambos enfoques guardan los mismos números, pero los objetos gastan memoria extra en cabeceras

y punteros que no aportan nada al problema. Lo que genera más trabajo al recolector de basura y más consumo de energía.

Que dos soluciones resuelvan el mismo problema no las hace igual de valiosas, porque en un proyecto de computación sostenible importa tanto el resultado como lo que cuesta producirlo.

Repositorio de Github Grupal:

https://github.com/freddyordonez/Portafolio-de-Estructura-de-Datos.git

7. Bibliografía

1. P. W. Bible and L. Moser, *An Open Guide to Data Structures and Algorithms*. PALNI Open Press, 2023. [Online]. Available: https://open.palni.org/
2. Mozilla Developer Network, "Memory management", *MDN Web Docs*, 2023. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Memory_Management
3. O. Levin, *Discrete Mathematics: An Open Introduction*, Autoeditado, 2024.